Part 1 — How a leader gets chosen

A node that wants to lead increases its term (like a round number) and asks others to vote for it. In this simple simulator, any online node counts as a yes vote. With 3 nodes online, 2 votes is a majority, so it becomes the leader.

The leader is the one coordinator: it handles client requests and copies the log to others. Terms keep election attempts in order and help spot old, out-of-date leaders.

Part 2 — Staying leader

The leader sends empty AppendEntries messages, called heartbeats. They carry no new data — they just say "I'm still here."

Followers restart their election timer whenever they hear from the leader. If heartbeats stop, followers may time out and start a new election. This simulator shows message delivery but has no real timers.

Part 3 — Leader fails, new leader elected

When the leader goes offline, it is no longer treated as the active leader. An online node can start a new election, raise its term, and become leader if a majority is available. The new term overrides the old leader's authority.

Part 4 — Copying the log

The leader creates a log entry with an index, a term, and a command, then sends it to online peers. The entry is marked committed only when a majority acknowledges it.

With 3 nodes, the leader plus 1 follower = 2 acknowledgements = majority. So progress continues even if one follower is down.

Part 5 — Keeping going when a node fails

A 3-node cluster needs 2 nodes for a majority. If one follower fails, the leader and the other follower still form a majority, so copying continues.

The failed follower means less redundancy, but progress does not stop.

Rule of thumb:

5 nodes → can lose 2 and still work.

3 nodes lost out of 5 → no majority, no progress.

Part 6 — When progress must stop

A 5-node cluster needs 3 nodes for a majority. If only 2 are online, there is no quorum.

The simulator then blocks new replication instead of pretending an entry was committed. This is on purpose: consistency comes first. Without a majority, the cluster gives up availability rather than risk inconsistency.

Part 7 — Network partitions

This scenario drops messages during a partition and delivers them again after the network is repaired.

During a full partition, everything stops because every message is dropped.

This network class uses a single global partition flag, so it cannot show separate majority and minority sides or fully model split-brain prevention.

Part 8 — Fixing a lagging follower

The follower starts with only 3 entries; the leader has 5. The recovery step copies the missing 4th and 5th entries, then compares the logs. They match, so the logs have converged in this example.

Part 9 — Catching operational problems

The hardening scenario checks basic safety assumptions:

- Is the election timeout at least a minimum value?
- Have there been too many elections (churn warning)?
- Do online nodes reach a majority?
- Are followers falling behind?
- Did state disappear when it shouldn't have?

These checks catch operational problems that hurt availability or reliability even when the protocol logic looks fine. The thresholds are examples and need tuning per deployment.

Part 10 — Making a Raft deployment safer

The added guards:

- reject unsafe timeout values,
- flag repeated elections,
- check majority health,
- warn about lagging followers,
- warn about suspicious lost state,
- write audit records for elections, heartbeats, replication, and commits.

These reduce the impact of operational failures. They do not replace Raft's core rules for voting, log matching, persistence, or commits.

For production you would also need durable state, reliable per-peer networking, authenticated communication, proper election timers, full Raft protocol handling, monitoring/observability, and fault-injection testing.